Data Cleaning Cheat Sheet

Solavise Women in Data (SWID) Programme • Python & Pandas

Data cleaning is the process of fixing problems in your dataset before you analyse it. Messy data leads to wrong results. Always clean first!

1 First Look at Your Data

Run these lines before you do anything else. They tell you what your data looks like.

See the first 5 rows	`df.head()`
See the last 5 rows	`df.tail()`
Rows and columns count	`df.shape`
Column names	`df.columns`
Column types and info	`df.info()`
Stats for number columns	`df.describe()`
See a random sample of rows	`df.sample(5)`

```
df.head() # first 5 rows
df.info() # column types + nulls
df.describe() # min, max, mean, etc.
```

2 Handling Missing Values

Missing values show up as NaN. You can find them, fill them, or remove them.

Find missing values	`df.isnull().sum()`
Drop rows with any NaN	`df.dropna()`
Drop only if ALL are NaN	`df.dropna(how='all')`
Fill NaN with a fixed value	`df.fillna(0)`
Fill NaN with column mean	`df['col'].fillna(df['col'].mean())`
Fill NaN with column median	`df['col'].fillna(df['col'].median())`
Fill NaN with most common val	`df['col'].fillna(df['col'].mode()[0])`

Tip: *Choose your fill strategy on purpose. Use the mean for numbers that are spread evenly. Use the median when there are outliers. Use the mode for categories or repeated values.*

3 Removing Duplicates

Duplicate rows can make your results look bigger than they really are.

Check for duplicates	`df.duplicated().sum()`
See duplicate rows	`df[df.duplicated()]`
Remove all duplicate rows	`df.drop_duplicates()`
Keep first occurrence only	`df.drop_duplicates(keep='first')`
Remove duplicates in one col	`df.drop_duplicates(subset=['col'])`

```
# Remove duplicates and save result
df = df.drop_duplicates()
print(df.shape) # check new size
```

4 Fixing Data Types

If a number column is saved as text, maths will not work on it. Fix data types early.

Convert to number	`pd.to_numeric(df['col'])`
Ignore errors when converting	`pd.to_numeric(df['col'], errors='coerce')`
Convert to date	`pd.to_datetime(df['col'])`
Convert to text	`df['col'].astype(str)`
Convert to integer	`df['col'].astype(int)`
Check all column types	`df.dtypes`

```
# Convert a date column
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
```

5 Cleaning Text

Text data is often messy. Extra spaces, mixed letter cases, and typos are very common.

Remove extra spaces	`df['col'].str.strip()`
Make all text lowercase	`df['col'].str.lower()`
Make all text uppercase	`df['col'].str.upper()`
Capitalise each word	`df['col'].str.title()`
Replace one value with another	`df['col'].replace('old','new')`
Replace using a dictionary	`df['col'].replace({'US':'USA'})`
Check for a word in text	`df['col'].str.contains('word')`

```
df['name'] = df['name'].str.strip()
df['city'] = df['city'].str.title()
df['status'] = df['status'].str.lower()
```

6 Renaming and Dropping Columns

Clean column names make your code easier to read and your analysis easier to share.

Rename one column	`df.rename(columns={'old':'new'})`
Rename multiple columns	`df.rename(columns={'a':'x','b':'y'})`
Drop a column	`df.drop(columns=['col'])`
Drop multiple columns	`df.drop(columns=['c1','c2'])`
Drop a row by index	`df.drop(index=0)`
Reset the row numbers	`df.reset_index(drop=True)`

```
# Rename and drop in one flow
df = df.rename(columns={'Nm':'Name'})
df = df.drop(columns=['unused_col'])
```

7 Spotting Outliers

Outliers are values that are way too high or too low. They can mislead your analysis.

See min and max	`df['col'].min() / df['col'].max()`
Filter rows above a value	`df[df['col'] > 1000]`
Filter using percentile	`df['col'].quantile(0.99)`
Remove extreme high values	`df[df['col'] < df['col'].quantile(0.99)]`
Remove extreme low values	`df[df['col'] > df['col'].quantile(0.01)]`

```
q_low = df['col'].quantile(0.01)
q_high = df['col'].quantile(0.99)
df = df[(df['col'] > q_low) &
(df['col'] < q_high)]
```

8 Saving Your Clean Data

Always save your cleaned dataset so you do not have to redo the work.

Save to CSV	`df.to_csv('clean_data.csv', index=False)`
Save to Excel	`df.to_excel('clean_data.xlsx', index=False)`
Load CSV back	`df = pd.read_csv('clean_data.csv')`
Load Excel back	`df = pd.read_excel('clean_data.xlsx')`

Quick Checklist Before Analysis